

操作系统第五次作业

1

(1)

- 先来先服务：1、2、3、4、5
- 短作业优先：2、4、3、5、1
- 非抢占式的优先数：2、5、1、3、4
- 轮转法（时间片为2）：1（剩余8）、2（完成）、3（完成）、4（完成）、5（剩余3）、1（剩余6）、5（剩余1）、1（剩余4）、5（完成）、1（剩余2）、1（完成）

(2)

•	先来先服务	周转时间	等待时间
	P1	10	0
	P2	11	10
	P3	13	11
	P4	14	13
	P5	19	14
	平均周转时间： $\frac{10+11+13+14+19}{5} = 13.4$		

•	短作业优先	周转时间	等待时间
	P2	1	0
	P4	2	1
	P3	4	2
	P5	9	4
	P1	19	9
	平均周转时间： $\frac{1+2+4+9+19}{5} = 7$		

•	非抢占式的优先数	周转时间	等待时间
	P2	1	0
	P5	6	1
	P1	16	6
	P3	18	16
	P4	19	18
	平均周转时间： $\frac{1+6+16+18+19}{5} = 12$		

轮转法	周转时间	等待时间
P2	3	2
P3	5	3
P4	6	5
P5	15	10
P1	19	9
平均周转时间: $\frac{3+5+6+15+19}{5} = 9.6$		

2

进程P会在第10、20、30、40、50个时间片被调度，第50个时间片完成，周转时间为250ms

3

(1)

当仅使用静态*nice*值作为优先数时，高*nice*值（低优先级）进程可能永远无法获得CPU。若系统持续存在低*nice*值（高优先级）进程，调度器每次都会选择这些低*nice*值进程执行，导致高*nice*值进程的*waitTime*无限增长但始终得不到调度，形成永久等待状态。

(2)

- 计算公式：

$$priority = nice + cpuTime - \lfloor \frac{waitTime}{k} \rfloor$$

($k = 10$ ，可以通过系数 k 调节权重平衡)

- 动态调节机制：

执行态进程：*cpuTime*每秒+1，*waitTime*置0

就绪态进程：*cpuTime*置0，*waitTime*每秒+1

- waitTime*的核心作用：

① 反衰减机制：*waitTime*的累加效应会逐步降低实际优先数，为长期等待的进程建立升优先级通道

② 公平性补偿：当*waitTime*超过阈值时， $\lfloor \frac{waitTime}{k} \rfloor$ 项将产生抵消*cpuTime*增长的作用

③ 状态转换激励：被调度执行后立即重置*waitTime*，防止优先级虚高

4

(1)

由于每个进程最多申请 k 个资源，则每个进程最多持有 $k - 1$ 个资源，最坏的情况下，每个进程都持有 $k - 1$ 个资源，则一共持有 $n(k - 1)$ 个资源。

剩余资源必须 ≥ 1 ，使得至少一个进程能完成。即： $m - n(k - 1) \geq 1$

变形可得： $nk \leq m + n - 1$

故各进程的最大需求量之和为 $m + n - 1$

(2)

使每个进程占有2台打印机，则共占有 $2k$ 台，要发生死锁，则 $2k + 1 > 8$ 即 $k \geq 4$

(3)

三个进程各占 $3 - 1, 4 - 1, 5 - 1$ 个资源，则至少有 $2 + 3 + 4 + 1 = 10$ 个资源可以避免死锁

5

- 进程之间的同步关系：
 - 同步关系是指多个进程为了完成一个共同的任务而需要协调它们的活动，确保按照特定的顺序执行。这种关系通常发生在进程需要合作完成一个任务，并且一个进程的输出是另一个进程的输入的情况下。
 - 同步的目的是确保进程之间的数据交换正确无误，并且所有相关的进程都能在适当的时机进行操作。
 - 例如，一个生产者-消费者问题中，生产者进程生成数据并将其放入缓冲区，消费者进程从缓冲区中取出数据进行处理。这里，生产者和消费者之间需要同步，以确保消费者不会在数据还未生成之前就尝试消费，同时也确保生产者不会在缓冲区已满时继续生产数据。
- 进程之间的互斥关系
 - 互斥关系是指多个进程在访问共享资源时，必须确保在同一时刻只有一个进程能够访问该资源。这种关系通常发生在多个进程需要访问同一临界资源（如共享变量、文件等）的情况下。
 - 互斥的目的是防止多个进程同时访问共享资源，从而避免数据不一致性和冲突。
 - 例如，考虑一个银行账户的余额更新操作，如果有多个进程同时尝试修改账户余额，可能会导致余额计算错误。因此，需要确保在任何时刻只有一个进程能够修改账户余额，这就是互斥。

6

(1)

先计算 $Need$ 矩阵

$$Need = Max - Allocation = \begin{bmatrix} 0 & 0 & 4 \\ 1 & 7 & 5 \\ 2 & 3 & 5 \\ 0 & 6 & 4 \\ 0 & 6 & 5 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 3 \\ 1 & 0 & 0 \\ 1 & 3 & 5 \\ 0 & 0 & 2 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 7 & 5 \\ 1 & 0 & 0 \\ 0 & 6 & 2 \\ 0 & 6 & 4 \end{bmatrix}$$

由矩阵可知，初始 $Work = Available = (1, 4, 0)$ 可满足 P_2 的要求，当其结束后释放资源， $Work = (2, 7, 5)$ ，可满足 P_0, P_1, P_3, P_4 任意一个，不会死锁，故为安全状态

(2)

若初始 $Work = Available = (0, 6, 2)$ ，可满足 P_0, P_3 的要求，这两个进程结束后释放资源， $Work = (0, 6, 7)$ ，可满足 P_4 ，结束后 $Work = (0, 6, 8)$ ，此时不满足剩下的任何一个需求，故为非安全状态。